

TP7 - Sistemas Operativos

Raid 0, 1 y 5 son los más usados.

1. Ninguno de los algoritmos de planificación de disco, excepto FCFS, es realmente equitativo (puede ocurrir inanición).

a) Explique por qué es cierta esta afirmación.

Esta afirmación es cierta ya que por empezar FCFS es el algoritmo más simple y, en principio, el más justo porque atiende las solicitudes en el orden en que llegan, sin dar prioridad a ninguna solicitud sobre otra. Esto significa que, si hay solicitudes pendientes, todas serán atendidas en su debido tiempo por más que no sea eficiente.

Por otro lado, si nos ponemos a analizar a los algoritmos SSTF, SCAN, Look, C-SCAN y C-Look. Comenzando por **SSTF** (Shortest Seek Time First) el cuál busca minimizar el tiempo de búsqueda, y el tiempo depende de la carga, no se puede evitar que haya inanición ya que el hecho de que pueda llevar a esperas largas por parte de algunos requerimientos lo hace inequitativo. Las solicitudes que estén muy alejadas de la cabeza del disco pueden quedarse esperando indefinidamente si siempre hay otras solicitudes más cercanas, lo que lleva a que nunca se les dé servicio.

Para el caso de **SCAN** o también conocido como el algoritmo del ascensor ya que el brazo del disco comienza en un extremo del disco y se mueve hacia el otro extremo, en su recorrido sirve todos los requerimientos hasta que llega al otro extremo donde se invierte el movimiento de la cabeza y continúa sirviendo los requerimientos. Entonces hay un caso donde sí se genera inanición, cuando permanentemente llegan pedidos al mismo disco y este está yendo en una dirección opuesta desde donde llegó el requerimiento, Esto, lleva a situaciones en las que ciertas solicitudes se queden esperando por largos períodos. Supongamos que el requerimiento R llega en esa posición y el brazo está en el paréntesis: —R—(->)-----.

Para el caso de **Look** el algoritmo se centra en que el brazo vaya tan tan lejos en cada dirección como el último requerimiento. Similar a SCAN, pero en LOOK la cabeza no va hasta el final del disco si no hay solicitudes allí. La inanición en este caso viene si muchas solicitudes se agrupan en una dirección de la cabeza, las solicitudes en la otra dirección pueden sufrir demoras significativas.

C-SCAN en vez de ascensor, una vez en el final vuelve a arrancar desde el principio. El problema de la inanición es trivial y similar al de SCAN. Luego el C-Look es lo mismo que Look pero nuevamente no da la vuelta por el mismo lugar sino que del final vuelve al inicio. El problema con la inanición es similar nuevamente.

El Drive de las respuestas del Silberschatz: En teoría, las nuevas solicitudes para la pista sobre la que reside actualmente la cabeza pueden llegar tan rápido como se atienden estas solicitudes.

b) Describa una forma de modificar el algoritmo de SCAN para garantizar que sea equitativa.

Una forma de modificar el algoritmo SCAN para hacerlo más equitativo sería introducir un mecanismo que garantice que las solicitudes más antiguas reciban prioridad. Esto podría lograrse haciendo que todas las solicitudes que superen un tiempo determinado sean 'forzadas' a la parte superior de la cola. Además, se establecería un indicador para cada solicitud que impida que nuevas solicitudes adelanten a estas solicitudes más antiguas.

Según la profesora en clase, tengo anotado que también una opción es enviar por ejemplo si tengo 20 requerimientos, a todos en una vez. Entonces ninguno sufre, y que no los mando de a uno y se evita el tema de la inanición.

Para implementar esta modificación en el algoritmo SSTF, sería necesario reorganizar el resto de la cola en función de la última de estas solicitudes 'antiguas', lo que garantizaría que todas las solicitudes sean atendidas de manera justa.

Otras opciones:

N-step-SCAN: Se divide la cola en subcolas de tamaño N, y se van atendiendo usando SCAN. Nuevas solicitudes se van añadiendo a otras colas. Si N es grande, se aproxima al SCAN clásico. Con $N = 1$, es FIFO.

FSCAN: Cuando se comienza, se dividen dos colas: Las solicitudes actuales, y las que llegarán. La segunda arranca vacía, pero se va llenando. Claramente, se revisa la cola que tiene las solicitudes actuales.

c) Explique por qué es importante el objetivo de la distribución equitativa en un sistema de tiempo compartido.

Es importante la distribución equitativa en un sistema de tiempo compartido, justamente porque los tiempos compartidos nos dicen que para cada usuario conectado al sistema, tienen que tener tiempos proporcionalmente parecidos de uso del CPU. Además la respuesta tiene que ser rápida, menor a un segundo por lo que si nos ponemos a trabajar con una planificación que no es equitativa no se va a poder lograr justamente el objetivo de un sistema de tal calibre. En conclusión, una distribución no equitativa nos daría tiempos largos de respuesta generando una mala experiencia para aquel usuario que sufre la inanición.

d) Proporcione tres o más ejemplos de circunstancias en las que sea importante que el sistema operativo no sea equitativo a la hora de satisfacer las solicitudes de E/S.

Algunos ejemplos donde conviene la inequidad:

- **Interacción en tiempo real con el usuario:** En situaciones donde el usuario está interactuando directamente con el sistema, como al escribir en un editor de texto, la prioridad debe darse a las solicitudes de E/S relacionadas con la entrada del teclado. Si el sistema tratara todas las solicitudes de E/S de manera equitativa, el proceso de

escritura podría verse interrumpido por otras tareas, afectando la experiencia del usuario y la fluidez de la interacción. En este caso, la escritura debe ser atendida con alta prioridad para evitar latencia y asegurar que las pulsaciones del teclado se procesen inmediatamente.

- **Procesos en tiempo real:** Los sistemas que ejecutan aplicaciones en tiempo real, como sistemas de control industrial o de telecomunicaciones, requieren que las solicitudes de E/S de esos procesos sean atendidas con mayor urgencia que otras tareas. Si el sistema operativo no prioriza las solicitudes de E/S de estos procesos, podrían producirse retrasos críticos, lo que afectaría la precisión y la confiabilidad del sistema en tiempo real. Aquí, la inequidad en el manejo de las solicitudes de E/S es esencial para cumplir con los requisitos estrictos de tiempo.
- **Manejo de memoria virtual (paginación y swapping):** En un sistema con memoria virtual, la paginación y el swapping (traslado de páginas entre la memoria principal y el disco) son operaciones críticas que deben tener alta prioridad sobre otras solicitudes de E/S, como la escritura de datos de los usuarios (más prioridad que el primer ejemplo por lo que puede ocurrir una pequeña contradicción). Si las operaciones de paginación se demoran debido a que otras solicitudes de E/S son atendidas primero, el rendimiento global del sistema podría verse gravemente afectado, incluso provocando fallos en la ejecución de los programas. Por lo tanto, el sistema operativo debe asegurar que estas tareas de manejo de memoria se realicen de manera eficiente y con mayor prioridad.

2. Se solicitan los siguientes cilindros del disco al manejador del disco: 10, 22, 20, 2, 40, 6 y 38, en ese orden. Una búsqueda tarda 6 mseg por cada cilindro desplazado. Decir cual es el tiempo de búsqueda necesario si se utilizan cada uno de los siguientes algoritmos:

- El primero en llegar es el primero en despacharse.
- Continuar con el cilindro más cercano.
- El algoritmo SCAN (posición inicial 21, en orden creciente).

HECHO EN HOJA DESPUÉS PEGAR

3. Suponga que el movimiento de cabezas de un disco con 200 tracks numerados de 0 a 199, está actualmente en el track 143. Si la cola de solicitudes ordenada es: 147, 91, 177, 94, 150, 102, 175, 130, 125. ¿Cuál es el número de cabezas necesarias para realizar estos requerimientos utilizando los siguientes algoritmos de planificación?:

- FCFS
- SSTF
- SCAN
- LOOK
- C-SCAN

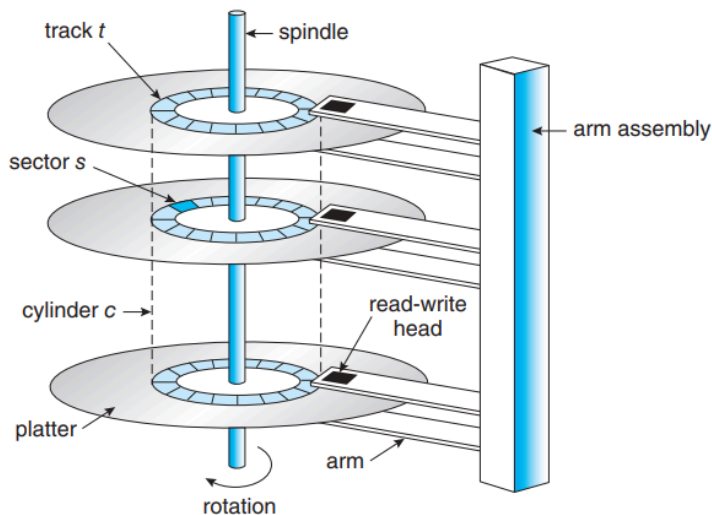
HECHO EN HOJA DESPUÉS PEGAR

4. Realice para los algoritmos FCFS, SSTF, SCAN y C-SCAN, el siguiente análisis, la cantidad de pistas recorridas para cada caso y el promedio de búsqueda para la siguiente secuencia de peticiones de pistas: 21, 129, 110, 186, 147, 41, 10, 64, 120.

Supóngase que la cabeza del disco está ubicada inicialmente sobre la pista 100 y se está moviendo en dirección decrecientes de números de pista. Haga el mismo análisis, pero suponga ahora que la cabeza del disco está moviéndose en direcciones crecientes de números de pista.

HECHO EN HOJA DESPUÉS PEGAR

5. Calcule cuánto espacio en disco (en sectores, pistas y superficie) será requerido para almacenar 250000 registros lógicos de 200 bytes, si el tamaño del sector es de 1024 bytes/sector, con 108 sectores/pista, 140 pistas por superficies y 12 superficies utilizables. Asuma que los registros no se pueden dividir entre dos sectores.



En base a la estructura de un disco convencional vamos a seguir los datos para calcular lo requerido.

Arrancando por lo de que cada registro lógico tiene un tamaño de 200 Bytes, si tenemos 250.000 registros entonces sacar el espacio necesario es fácil (en bytes):

Espacio necesario = 250.000 * 200 Bytes
Espacio necesario = **50.000.000 Bytes**

Luego tenemos que cada sector tiene un tamaño total de 1KB o 1024 Bytes. Este tamaño es mucho más grande que un registro lógico (200 Bytes) por lo que metiendo un registro lógico en un sector ocurrirá fragmentación interna (quedando más de 800 Bytes libres). Sin embargo no nos dice que por cada sector puede haber un sólo registro, sólo nos dice que uno no puede estar dividido entre dos sectores. Por lo que podemos hacer fácilmente el cálculo de cuántos registros lógicos pueden entrar en un sector.

Cantidad reg. log. x sector = $1024 \text{ Bytes} / 200 \text{ Bytes} = 5,12 \rightarrow 5$ registros lógicos

Una vez sabido cuántos registros hay en un sector calculamos la cantidad que vamos a tener de los mismos:

Cantidad sectores = $250.000 \text{ registros} / 5 = \mathbf{50.000 \text{ sectores}}$

Una pista es un círculo o una línea en el disco que se encuentra en una capa específica de la superficie del disco. En el gráfico está marcado como "platter", y según el enunciado podemos tener 108 sectores por pista, es decir 108 divisiones de cuadraditos por disco

como se ve en la imagen. Teniendo la cantidad de sectores entonces nos resulta también fácil calcular la cantidad de pistas necesarias:

$$\text{Cantidad pistas} = 50.000 \text{ sectores} / 108 = 462,9 \rightarrow \mathbf{463 \text{ pistas}}$$

Con superficie por otro lado, se refiere a cada una de las caras de un disco. Los discos duros tradicionales tienen más de una superficie de almacenamiento (por ejemplo, dos superficies por disco, una superior y una inferior). Las superficies están cubiertas por un material magnético (en discos duros tradicionales) que permite almacenar datos. Por enunciado sabemos que cada disco tiene 140 pistas por superficie, lo que significa que hay 140 líneas concéntricas de datos en cada cara del disco. En un disco con múltiples superficies, las superficies están apiladas. Si un disco tiene 12 superficies utilizables, cada una tiene su propia serie de pistas. Pero estos son datos extra, calculemos entonces la cantidad de superficies que tenemos:

$$\text{Cantidad superficies} = 463 \text{ pistas} / 140 = 3,3 \rightarrow \mathbf{4 \text{ superficies}}$$

En conclusión serán requeridos:

- 50.000 sectores
- 463 pistas
- 4 superficies

6. La asignación adyacente de los archivos produce fragmentación en el disco. ¿Es esta fragmentación interna o externa?

La asignación adyacente de archivos produce fragmentación **externa**, no interna. En la asignación adyacente, los archivos se almacenan de manera contigua en el disco, lo que implica que el espacio libre del disco puede dividirse en fragmentos pequeños a medida que los archivos son creados y eliminados. Esto se asemeja a la fragmentación externa en la memoria principal, como ocurre con el swapping o la segmentación pura, donde el espacio libre se divide en fragmentos dispersos.

La fragmentación externa se produce cuando hay huecos en el disco entre los archivos asignados (probablemente por haber borrado otros archivos que se pusieron en su momento), lo que dificulta la utilización eficiente del espacio disponible. Este problema surge cuando el espacio libre del disco se fragmenta en trozos pequeños, y el espacio contiguo más grande disponible no es suficiente para satisfacer una nueva solicitud de almacenamiento. Así, aunque haya suficiente espacio libre en total, este espacio no puede aprovecharse eficientemente debido a su distribución en fragmentos pequeños.

7. El rendimiento de un sistema de archivos depende de una manera crítica de la cantidad de hit en el caché. Si se toma 1 mseg satisfacer una solicitud del cache y 40 mseg satisfacer una solicitud si se requiere la lectura del disco, de una fórmula para el tiempo promedio necesario para cumplir con una solicitud si la razón de hit es h.

$$1 \text{ mseg} + (1 - h) * 40 \text{ mseg}$$

8. Compare el throughput de los algoritmos SCAN y C-SCAN, asumiendo una distribución uniforme de solicitudes.

Antes que nada estaría bueno explicar a qué se refiere throughput en este contexto, el cual se refiere a la eficiencia de los algoritmos en procesar solicitudes de E/S. Un mayor throughput implica que un algoritmo puede procesar más solicitudes en un período de tiempo más corto.

Una vez explicado esto para comparar estos dos algoritmos me voy a apoyar en una investigación realizada entre los algoritmos C-Look y Look los cuáles son muy parecidos pero la optimización de SCAN Y C-SCAN. En estos se demostró que C-LOOK tiene un tiempo de respuesta promedio, apenas un porcentaje más alto que LOOK, pero C-LOOK tiene una varianza significativamente menor en el tiempo de respuesta para cargas de trabajo medias y pesadas. La razón intuitiva para la diferencia de la varianza es que LOOK (y SCAN) tiende a favorecer las solicitudes cerca de los cilindros centrales, mientras que las versiones circulares no tienen este desequilibrio. La razón intuitiva para el tiempo de respuesta más lento de C-LOOK (y C-SCAN) es la búsqueda "circular" desde un extremo del disco hasta la solicitud más lejana en el otro extremo. Esta búsqueda no satisface ninguna solicitud. Solo causa una pequeña degradación del rendimiento porque la dependencia de raíz cuadrada del tiempo de búsqueda con respecto a la distancia implica que una búsqueda larga no es terriblemente costosa en comparación con las búsquedas de longitud moderada.

9. La fragmentación en un disco puede ser eliminada mediante la compactación de información. ¿Cómo se puede reubicar los archivos?. Dos razones por las cuales la reubicación y compactación de archivos es evitada.

Silberschatz - Chapter 14 - File-System Implementation - Page 572

Antes que nada la fragmentación que puede ser compactada es la externa como el caso que se preguntó en el inciso 7. Entonces la manera de realizar esto es copiando todo un sistema de archivos a otro dispositivo. El dispositivo original se libera completamente, creando un gran espacio libre contiguo. Luego, copiamos los archivos de vuelta al dispositivo original, asignando espacio contiguo a partir de este gran hueco. Este esquema efectivamente compacta todo el espacio libre en un solo espacio contiguo, resolviendo el problema de la fragmentación.

Hay varias razones por lo que son evitadas, ya que la manera en que afecta la fragmentación externa depende de la cantidad total de almacenamiento en el disco y del tamaño promedio de los archivos. Sin embargo, para la solución que dimos arriba la primer razón es que el costo de esta compactación es el tiempo, sin embargo, y el costo puede ser particularmente alto para dispositivos de almacenamiento grandes. Compactar estos dispositivos puede llevar horas y puede ser necesario hacerlo semanalmente. Algunos sistemas requieren que esta función se realice fuera de línea, con el sistema de archivos desmontado. Durante este tiempo de inactividad, generalmente no se permite la operación normal del sistema, por lo que se evita la compactación a toda costa en máquinas de producción. La mayoría de los sistemas modernos que necesitan desfragmentación pueden

realizarla en línea durante las operaciones normales del sistema, pero la penalización en el rendimiento puede ser considerable.

Otra de las razones es que con la asignación contigua determinar cuánto espacio se necesita para un archivo es complejo. Cuando se crea el archivo, debe encontrarse y asignarse la cantidad total de espacio que necesitará. ¿Cómo sabe el creador (programa o persona) el tamaño del archivo que se va a crear? En algunos casos, esta determinación puede ser bastante simple (por ejemplo, al copiar un archivo existente). Sin embargo, en general, el tamaño de un archivo de salida puede ser difícil de estimar.

10. ¿La velocidad de rotación de un disco mejora su tiempo de acceso? ¿por qué?

Silberschatz - 11.1.1 Hard Disk Drives

Sí, la velocidad de rotación de un disco mejora su tiempo de acceso. Esto se debe a que el tiempo de acceso tiene dos componentes: el tiempo de búsqueda y la latencia rotacional. La latencia rotacional es el tiempo que tarda el disco en girar hasta que el sector deseado llega a la cabeza de lectura/escritura. Una mayor velocidad de rotación (más RPM) reduce este tiempo, ya que los sectores pasan más rápido frente a la cabeza de lectura/escritura, mejorando así el tiempo de acceso en general. Sin embargo, la velocidad de rotación no afecta el tiempo de búsqueda.

T = transfer time

b = number of bytes to be transferred

N = number of bytes on a track

r = rotation speed, in revolutions per second

11.5 / DISK SCHEDULING 489

Thus, the total average access time can be expressed as

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

where T_s is the average seek time.

Info en el Stallings.

11. La latencia no es considerada en los algoritmos de planificación de disco. ¿Cómo modificaría el SSF, SCAN y C-SCAN para que incluyan una optimización en la latencia?

La mayoría de los discos no exportan su información de posición de rotación al host. Incluso si lo hicieran, el tiempo para que esta información llegue al programador estaría sujeto a imprecisión y el tiempo consumido por el programador es variable, por lo que la información de la posición de rotación sería incorrecta.

12. ¿Dónde ubicaría físicamente los directorios en un disco?

Una estrategia común es ubicar los directorios al comienzo del disco o partición, cerca del área inicial donde el sistema de archivos está gestionado. Esto se hace para que los directorios principales sean fácilmente accesibles sin tener que buscar en áreas dispersas del disco. Dado que los directorios a menudo contienen información sobre otros archivos, su ubicación en una parte accesible de la estructura del disco permite que la gestión de archivos sea más eficiente.

En algunos sistemas de archivos, los directorios pueden estar dispersos a lo largo del disco, con entradas que apuntan a los bloques de disco donde se encuentran los archivos. Esto es característico de sistemas de archivos como FAT, donde las entradas de los directorios se distribuyen en varias ubicaciones dentro del espacio del disco. Este enfoque puede afectar la eficiencia, dependiendo de cómo se gestione la fragmentación.

En otros casos, especialmente en sistemas de archivos que favorecen la organización contigua (como los sistemas basados en inodos), los directorios pueden estar ubicados de manera contigua o junto a los bloques de datos correspondientes. Por ejemplo, en un sistema basado en inodos, el directorio podría estar físicamente en los primeros bloques del disco, con un puntero hacia las estructuras de inodos que describen los archivos.

También es posible reservar áreas separadas del disco para almacenar exclusivamente los directorios, con el fin de evitar la fragmentación interna que podría ralentizar las búsquedas. Este enfoque es común en sistemas de archivos más avanzados, como ext4 o NTFS, que utilizan estrategias para minimizar la fragmentación de los directorios.

Todos estos temas, sin embargo, van a ser tratados con mayor profundidad en el próximo TP. Agregado: el Mati en la chori-paneada me dijo que físicamente estaría en el centro del disco, ya que ahí estarían ubicados al comienzo del disco como comentamos en la primera estrategia.

13. ¿Es necesario utilizar algún algoritmo de planificación para un disco RAM? ¿Por qué?

No es necesario ya que la memoria RAM es una memoria volátil, donde todos los accesos a los datos se realizan de manera instantánea, sin necesidad de mover ningún componente físico. Además estos accesos son random como lo dice su nombre, porque son ampliamente superiores en velocidad respecto a los discos los cuáles si necesitan de ellos.